

Patrones de diseño

¿Qué son los patrones de diseño?

Un patrón de diseño es una solución probada y reutilizable a un problema que aparece una y otra vez cuando programamos. No es código que puedas copiar y pegar directamente en tu proyecto, sino más bien una receta o esquema que te dice cómo organizar tu código para resolver ese problema de la mejor manera posible.

El concepto fue popularizado en 1994 por un libro llamado *"Design Patterns"*, escrito por cuatro ingenieros conocidos como la Gang of Four: Erich Gamma, Richard Helm, Ralph Johnson y John Vlissides. En ese libro catalogaron 23 patrones, que siguen siendo la referencia hoy en día, y los dividieron en tres grandes familias:

- **Creacionales:** se ocupan de cómo se crean los objetos. En vez de crearlos directamente de cualquier manera, estos patrones te dan formas más controladas y flexibles de hacerlo.
- **Estructurales:** se ocupan de cómo se organizan y relacionan las clases entre sí para formar estructuras más grandes.
- **De comportamiento:** se ocupan de cómo se comunican los objetos entre ellos y cómo se reparten el trabajo.

¿Cuándo se usan los patrones de diseño?

Los patrones de diseño no son algo que apliques desde el primer momento porque sí. Se usan cuando detectas que tu código tiene algún problema concreto. Algunos síntomas de que necesitas un patrón son:

- **Tu código es muy rígido:** cambiar una cosa pequeña obliga a modificar muchos ficheros a la vez, con el riesgo de romper algo que antes funcionaba.
- **Tus clases están demasiado unidas entre sí:** si la clase A necesita conocer demasiados detalles internos de la clase B, cualquier cambio en B afecta a A. Esto se llama acoplamiento, y es malo.
- **Estás repitiendo la misma lógica en varios sitios:** copiar y pegar código es una señal de alarma. Si algo cambia, tendrás que cambiarlo en todos los sitios donde lo pegaste.
- **Añadir funcionalidad nueva es un dolor de cabeza:** si cada vez que quieres añadir algo tienes que reescribir código que ya funcionaba, algo está mal diseñado.
- **Varios objetos necesitan enterarse de lo que le pasa a otro:** por ejemplo, cuando un usuario hace una acción y varias partes de la aplicación tienen que reaccionar. Sin un patrón, esto se convierte en un lío de llamadas cruzadas.

¿Porqué son útiles los patrones de diseño?

Los patrones de diseño aportan varias ventajas muy concretas:

Vocabulario común entre programadores. Si trabajas en equipo y le dices a un compañero "aquí deberíamos usar un Observer", los dos sabéis de qué estáis hablando sin necesidad de explicar todo el diseño desde cero. Es como tener un idioma compartido.

El código es más fácil de mantener. Cuando sigues un patrón, el código queda organizado de una forma que otras personas pueden entender más rápido. No tienes que adivinar por qué está escrito así, porque sigue una estructura conocida.

Evitar reinventar la rueda. Los patrones son soluciones que ya han sido probadas en miles de proyectos reales. En vez de intentar resolver el problema desde cero y cometer los errores que otros ya cometieron, aplicas directamente lo que se sabe que funciona.

El código es más fácil de ampliar. Muchos patrones están diseñados siguiendo el principio Open/Closed: el código debe estar abierto para añadir cosas nuevas, pero cerrado para modificar lo que ya funciona. Esto significa que puedes añadir nueva funcionalidad sin tocar el código existente, reduciendo el riesgo de introducir errores.

¿Son diseño, arquitectura o microarquitectura?

Para responder bien a esta pregunta hay que entender que el software se puede analizar a distintos niveles, como si fueras acercando o alejando el zoom:

Arquitectura es el nivel más alto. Aquí se deciden cosas cómo: ¿va a ser una aplicación web o de escritorio? ¿Los diferentes módulos van a ser microservicios independientes o un solo bloque? ¿Cómo se comunican entre sí? Ejemplos: arquitectura cliente-servidor, microservicios, MVC. Estas decisiones afectan a todo el sistema.

Diseño es un nivel intermedio. Aquí se decide cómo se organiza cada módulo o subsistema por dentro: qué clases va a tener, cómo se divide la responsabilidad entre ellas, cómo fluyen los datos. Afecta a partes concretas del sistema, no a todo.

Microarquitectura es el nivel más bajo y detallado. Aquí es donde viven los patrones de diseño. Se ocupan de cómo se relacionan unas pocas clases concretas entre sí para resolver un problema específico. No deciden cómo se despliega el sistema ni cómo se comunican los servicios, sino algo mucho más local: cómo está construida internamente una parte pequeña del código.

Por tanto, los patrones de diseño son microarquitectura. Actúan a escala pequeña, sobre clases y objetos individuales. Sin embargo, aunque su alcance es local, su impacto es grande: un buen uso de patrones hace que el código de bajo nivel sea limpio, flexible y fácil de entender, lo cual beneficia a todo el sistema por encima.